EE5203 L02 Lab Guide

Process eight sensor values with decisions and loops - Basri Erdogan

Mission: Copy your working L01 project and process eight simulated sensor values in one `main.c` file. Clamp invalid values, calculate an average, count high values, and select a numeric status. Prove every result in the debugger.

Prepare before class

- ☐ Bring your working L01 Nucleo-F401RE project and USB data cable.
- ☐ Review assignment, arithmetic, `if/else`, `for`, and array indices.
- ☐ Complete the entry ticket below.

```c
int x = 3;
x = x + 2;
x++;
```

1. What is the final value of `x`?
2. Which symbol compares two values: `=` or `==`?
3. What are the valid indices of `int values[4]`?

Learning objectives

By checkoff time, you can:

1. trace assignments and arithmetic statements in order;
2. use comparisons in `if/else` decisions;
3. trace a `for` loop from its first to last valid counter value;
4. process every element of an eight-element array;
5. use debugger evidence to justify a prediction.

Hardware and files

Item	Required value
Board	Nucleo-F401RE
Tools	STM32CubeMX + VS Code
Starting point	Working copy of the L01 project
Project name	EE5203_L02_<studentnumber>
File you edit	Generated `Core/Src/main.c`
New source/header files	None
Input	Eight fixed `uint16_t` values
Evidence	Build console, breakpoint, Watch/Expressions

Keep all edits to generated `main.c` inside the appropriate USER CODE regions.

Rules

The eight fixed input values are declared in Checkpoint B. Rules:

- valid sensor values are `0` through `4095`;
- a value above `4095` must become `4095`;
- a high sample is strictly greater than `2500`;
- status code is `-1` below `1500`, `1` above `3000`, otherwise `0`.

Warm-up (first 15 minutes)

Before editing, trace this loop on paper:

```c
int total = 0;

for (int i = 0; i < 3; i++) {
    total += i;
}
```

Write the value of `i` and `total` after every repetition, name the initialization / condition / update parts, and say why `i <= 3` would be wrong for a three-element array.

Checkpoint A - Copy and build the project

1. Copy the L01 project and rename it EE5203_L02_<studentnumber>.
2. Build the unchanged copy; confirm zero errors.
3. Locate `USER CODE BEGIN 2` in `main.c`.

Evidence: Show the copied project name and the successful build.

Checkpoint B - Declare the data and result variables

Inside `USER CODE BEGIN 2`, before the generated `while (1)`, add:

```
uint16_t samples[8] = {
    850, 1250, 4095, 4200,
    2050, 900, 3100, 2800
};

int sum = 0;
int average = 0;
int high_count = 0;
int invalid_count = 0;
int status_code = 0;
```

For each declaration, be able to name the type, the starting value, and the valid array indices.

Evidence: Build with zero errors and show the declarations.

Checkpoint C - Process every array element

Add this loop after the declarations:

```
for (int i = 0; i < 8; i++) {
    if (samples[i] > 4095) {
        samples[i] = 4095;
        invalid_count++;
    }

    sum += samples[i];

    if (samples[i] > 2500) {
        high_count++;
    }
}
```

Before running, answer:

1. Which input value will be changed?
2. What will replace it?
3. Which indices will the loop visit?
4. Why is `i <= 8` invalid?

Evidence: Break inside the loop, single-step two repetitions, and show `i`, `samples[i]`, `sum`, and `high_count`.

Checkpoint D - Calculate and classify the result

After the loop, add:

```
average = sum / 8;

if (average < 1500) {
    status_code = -1;
} else if (average > 3000) {
    status_code = 1;
} else {
    status_code = 0;
}
```

The numeric status is deliberate. C strings and character arrays are introduced in Lecture 3.

Before debugging, calculate the expected:

Result	Prediction
`samples[3]` after clamping	
`invalid_count`	
`high_count`	
`sum`	
`average`	
`status_code`	

Evidence: Break after the `if/else` statement and compare every observed value with the prediction.

Checkpoint E - Make one controlled change

Change only the first sample from `850` to `3500`. Predict which results change and which do not, debug again, and explain any wrong prediction. Restore the original value afterwards.

Evidence: Show the revised prediction and the debugger result.

Common problems

Symptom	First check
Unknown type at build	`#include <stdint.h>` is present
Result too large / loop never stops	`sum` starts at zero; `i++` is present
Memory changes unexpectedly	condition is `i < 8`, not `i <= 8`
Change has no effect	rebuild and restart the debug session

Acceptance checklist

- ☐ Project builds with zero errors.
- ☐ Only `main.c` was edited.
- ☐ Array indices stay between `0` and `7`.
- ☐ `4200` is replaced by `4095`.
- ☐ Sum, average, counts, and status match the prediction.
- ☐ The controlled input change is demonstrated.
- ☐ Every submitted screenshot shows readable variables.

Individual oral check

Be prepared to answer one question and make one live change:

- What is the difference between `=` and `==`?
- Why does the loop use `i < 8`?
- What value does `samples[i]` select during a repetition?
- Why must `sum` start at zero?
- Change one threshold and predict the result before running.

Submit

Submit `main.c`, the completed prediction table, one build-console screenshot, one debugger screenshot of the final array and results, and three sentences explaining the controlled change.